

Отчет по лабораторной работе №3

Модификации градиентного спуска

Выполнили:

Новиков Алексей Георгиевич, группа М3233

Душкин Александр Алексеевич, группа М3233

Санкт-Петербург

2026

Содержание

1	Постановка задачи	2
2	Используемые модели задач	2
3	Описание реализованных методов	3
3.1	Momentum	3
3.2	Метод Нестерова	4
3.3	AdaGrad	4
3.4	RMSProp	5
3.5	AdaDelta	5
3.6	Adam	6
4	Структура программной реализации	6
5	Результаты вычислительных экспериментов	7
5.1	Квадратичные функции	7
5.2	Чувствительность к параметрам	8
5.3	Траектории на квадратичных функциях	11
5.4	Сложные функции	12
5.5	Траектории на сложных функциях	14
6	Сводная таблица эксперимента	16
7	Общий вывод	16

1 Постановка задачи

Цель работы — реализовать и исследовать модификации градиентного спуска, которые используют инерцию, накопление статистики градиентов или адаптивное масштабирование шага по координатам.

В работе реализованы:

1. Momentum;
2. Nesterov accelerated gradient;
3. AdaGrad;
4. RMSProp;
5. AdaDelta;
6. Adam.

Во всех экспериментах использовалась фиксированная точность

$$\varepsilon = 10^{-8},$$

а критерием остановки была норма градиента:

$$\|\nabla f(x_k)\| \leq \varepsilon.$$

Промежуточный вывод. Третья лабораторная продолжает вторую: базовое движение по антиградиенту заменяется правилами, которые используют память о прошлых шагах или статистику прошлых градиентов. Поэтому основной объект исследования смещается к гиперпараметрам: именно они определяют, насколько сильно метод доверяет текущему градиенту, прошлому направлению и накопленным масштабам по координатам.

2 Используемые модели задач

Модели задач переиспользуются из лабораторной работы №2:

- хорошо обусловленная квадратичная функция

$$f(x, y) = \frac{1}{2}(x^2 + y^2);$$

- плохо обусловленная квадратичная функция

$$g(x, y) = \frac{1}{2}x^2 + 50y^2;$$

- функция Розенброка

$$r(x, y) = (1 - x)^2 + 100(y - x^2)^2;$$

- функция Экли;
- функция Химмельблау.

Для квадратичных функций использовалась стартовая точка $(-2, -2)$. Для сложных функций в конфигурации задавались несколько стартовых точек, чтобы проверить чувствительность методов к области притяжения.

Промежуточный вывод. Повторное использование моделей из второй лабораторной делает сравнение корректным: меняется не задача, а только правило построения следующего приближения. Это важно методически, потому что Momentum, RMSProp и Adam отличаются не направлением базового градиента как таковым, а способом преобразовать его в обновление с учетом истории. На одних и тех же поверхностях можно увидеть, какие трудности решает память метода, а какие остаются из-за геометрии функции.

3 Описание реализованных методов

3.1 Momentum

Метод Momentum добавляет инерционный член:

$$v_t = \beta v_{t-1} + \alpha \nabla f(x_t),$$

$$x_{t+1} = x_t - v_t.$$

Параметр β отвечает за сохранение предыдущего направления движения.

Промежуточный вывод. Инерция ускоряет движение вдоль устойчивого направления, потому что новый шаг складывается не только из текущего градиента, но и из накопленной скорости. Если несколько градиентов подряд имеют близкое направление, скорость растет и метод быстрее проходит пологие участки. Но та же логика может навредить около минимума или в узкой долине: накопленная скорость продолжает тянуть точку вперед, даже когда текущий градиент уже требует торможения или поворота.

3.2 Метод Нестерова

В методе Нестерова градиент вычисляется в точке предпросмотра:

$$\begin{aligned}\tilde{x}_t &= x_t - \beta v_{t-1}, \\ v_t &= \beta v_{t-1} + \alpha \nabla f(\tilde{x}_t), \\ x_{t+1} &= x_t - v_t.\end{aligned}$$

Промежуточный вывод. Метод Нестерова пытается заранее оценить, куда приведет инерция, поэтому градиент вычисляется не в текущей точке, а в точке предпросмотра. Такая структура делает корректировку направления более ранней: метод видит последствия накопленной скорости до фактического обновления. За это он платит дополнительным вычислением градиента на каждой итерации, что видно в счетчиках вызовов.

3.3 AdaGrad

AdaGrad накапливает сумму квадратов градиентов:

$$\begin{aligned}s_t &= s_{t-1} + \nabla f(x_t)^2, \\ x_{t+1} &= x_t - \alpha \frac{\nabla f(x_t)}{\sqrt{s_t} + \varepsilon_{\text{num}}}.\end{aligned}$$

Промежуточный вывод. AdaGrad уменьшает эффективный шаг по координатам, где градиенты часто велики, потому что знаменатель содержит накопленную сумму квадратов градиента. Это полезно при разных масшта-

бах координат: направление автоматически нормируется по истории. Однако накопитель только растёт, поэтому на длинных детерминированных запусках эффективный шаг монотонно уменьшается и метод может почти остановиться до достижения требуемой точности.

3.4 RMSProp

RMSProp заменяет полную сумму квадратов экспоненциальным средним:

$$s_t = \rho s_{t-1} + (1 - \rho) \nabla f(x_t)^2,$$

$$x_{t+1} = x_t - \alpha \frac{\nabla f(x_t)}{\sqrt{s_t} + \varepsilon_{\text{num}}}.$$

Промежуточный вывод. Экспоненциальное сглаживание предотвращает бесконечный рост накопителя, потому что старые квадраты градиента постепенно забываются с коэффициентом ρ . В отличие от AdaGrad, RMSProp масштабирует шаг по недавней, а не по всей накопленной истории. Поэтому метод лучше адаптируется, когда масштаб градиента меняется по мере движения к минимуму.

3.5 AdaDelta

AdaDelta использует экспоненциальные средние квадратов градиентов и квадратов обновлений:

$$E[g^2]_t = \rho E[g^2]_{t-1} + (1 - \rho) g_t^2,$$

$$\Delta x_t = - \frac{\sqrt{E[\Delta x^2]_{t-1} + \varepsilon_{\text{num}}}}{\sqrt{E[g^2]_t + \varepsilon_{\text{num}}}} g_t,$$

$$E[\Delta x^2]_t = \rho E[\Delta x^2]_{t-1} + (1 - \rho) \Delta x_t^2.$$

Промежуточный вывод. Метод не требует явного глобального шага, потому что масштаб обновления определяется отношением накопленного среднего квадратов прошлых шагов к среднему квадратов градиентов. Такая самонормировка делает AdaDelta устойчивым к выбору learning rate, но в рассматриваемых задачах приводит к консервативным обновлениям: если средний размер прошлых шагов мал, новые шаги также остаются малыми, и метод часто доходит до лимита итераций.

3.6 Adam

Adam объединяет Momentum и RMSProp:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Используется коррекция смещения:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Обновление:

$$x_{t+1} = x_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon_{\text{num}}}.$$

Промежуточный вывод. Adam является наиболее универсальным из рассмотренных адаптивных методов, потому что объединяет две разные идеи: первый момент m_t задает сглаженное направление движения, а второй момент v_t нормирует шаг по координатам. Коррекция смещения особенно важна в начале оптимизации, когда накопители еще малы и без нее первые обновления были бы систематически искажены.

4 Структура программной реализации

Лабораторная работа построена поверх той же библиотеки `optimlib`, что и лабораторная №2. Ключевые элементы:

- `AdaptiveOptimizer` — базовый класс для методов с внутренним состоянием;
- `_reset_state()` — инициализация буферов перед каждым запуском;
- `_adaptive_direction(...)` — метод, в котором конкретный оптимизатор строит вектор обновления;
- `Momentum`, `Nesterov`, `AdaGrad`, `RMSProp`, `AdaDelta`, `Adam` — реализации методов;

- `OptimizationExperiment` — перебор функций, стартовых точек и сеток параметров;
- `plot_param_sensitivity` — построение line-графиков и heatmap чувствительности.

Сетки гиперпараметров задаются в `lab3/config.yaml`. Для Momentum и Nesterov перебираются α и β , для RMSProp — α и ρ , для AdaGrad — α , для AdaDelta — ρ , для Adam — α , β_1 и β_2 .

Промежуточный вывод. Общая архитектура позволила добавить шесть новых оптимизаторов без изменения кода функций и экспериментального раннера, потому что различия методов локализованы в построении вектора обновления. Базовый класс отвечает за цикл оптимизации, критерии остановки и историю, а конкретные методы хранят только свои состояния: скорость, накопитель квадратов градиента или моменты Adam. Такая структура хорошо соответствует математике методов с памятью.

5 Результаты вычислительных экспериментов

5.1 Квадратичные функции

В таблице приведены лучшие наборы параметров по числу итераций на квадратичных функциях.

Таблица 1: Лучшие параметры на квадратичных функциях при $\varepsilon = 10^{-8}$

Функция	Метод	α	β/ρ	β_1	β_2	Итерации
f	Momentum	10^{-2}	0.85	—	—	182
f	Nesterov	10^{-2}	0.85	—	—	202
f	AdaGrad	10^{-3}	—	—	—	1500
f	RMSProp	10^{-2}	0.85	—	—	238
f	AdaDelta	—	0.50	—	—	1500
f	Adam	10^{-2}	—	0.70	0.90	289
g	Momentum	10^{-2}	0.85	—	—	260
g	Nesterov	10^{-2}	0.85	—	—	202
g	AdaGrad	10^{-3}	—	—	—	1500
g	RMSProp	10^{-2}	0.85	—	—	239
g	AdaDelta	—	0.50	—	—	1500
g	Adam	10^{-2}	—	0.70	0.95	362

Промежуточный вывод. На квадратичных функциях наиболее быстрыми оказались Momentum, Nesterov и RMSProp. У инерционных методов это объясняется накоплением направления: на квадратичной поверхности градиенты меняются достаточно предсказуемо, поэтому скорость помогает проходить пологие участки. RMSProp выигрывает за счет координатного масштабирования, полезного особенно на плохо обусловленной функции. AdaGrad и AdaDelta в выбранной сетке часто не достигали точности до лимита, потому что их механизмы уменьшения эффективного шага делали обновления слишком осторожными.

5.2 Чувствительность к параметрам

Для методов с двумя параметрами строились heatmap, где цвет отражает число итераций. Для методов с одним параметром строились line-графики.

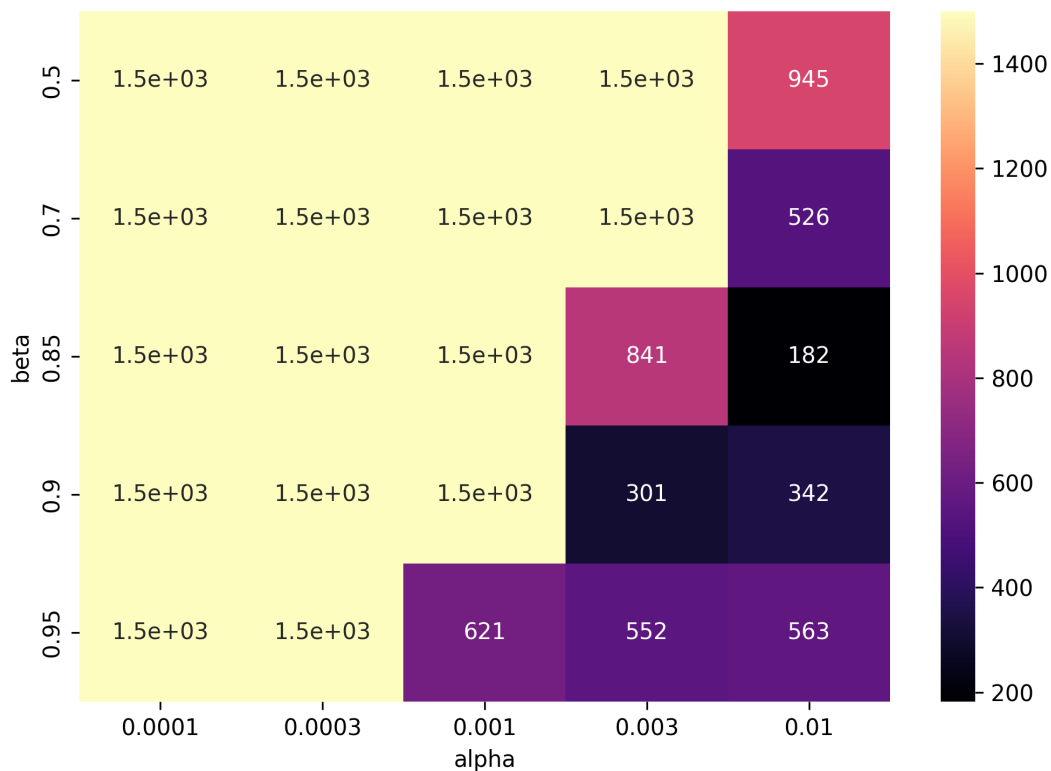


Рис. 1: Чувствительность Momentum к параметрам α и β

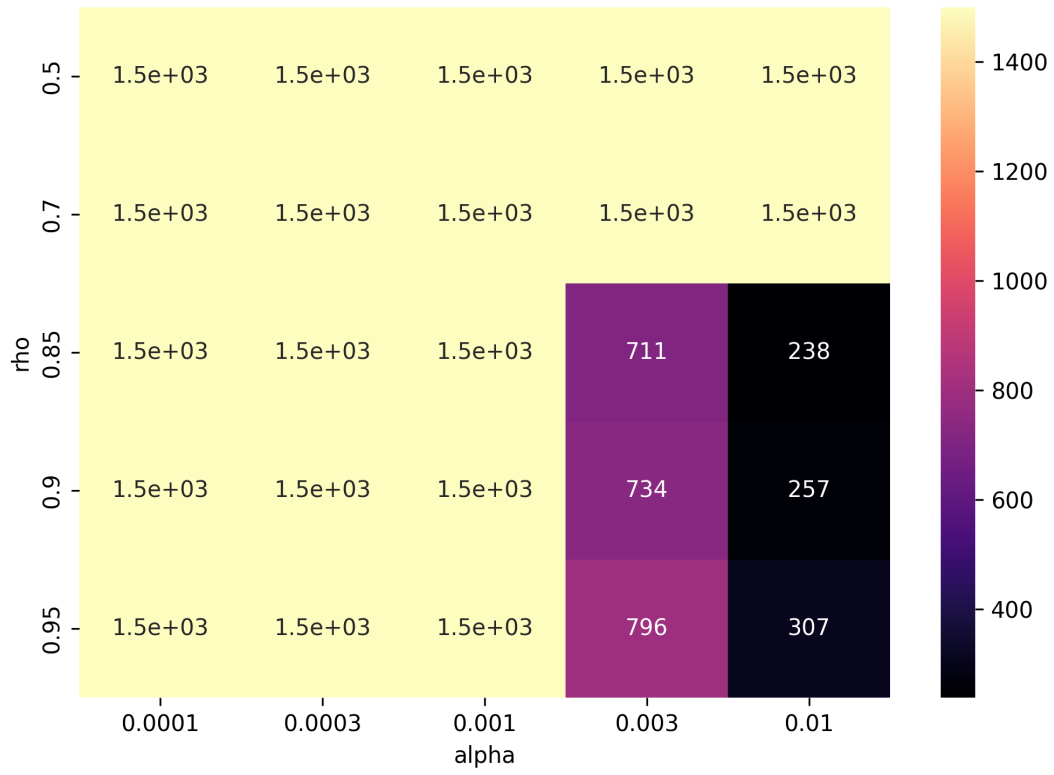


Рис. 2: Чувствительность RMSProp к параметрам α и ρ

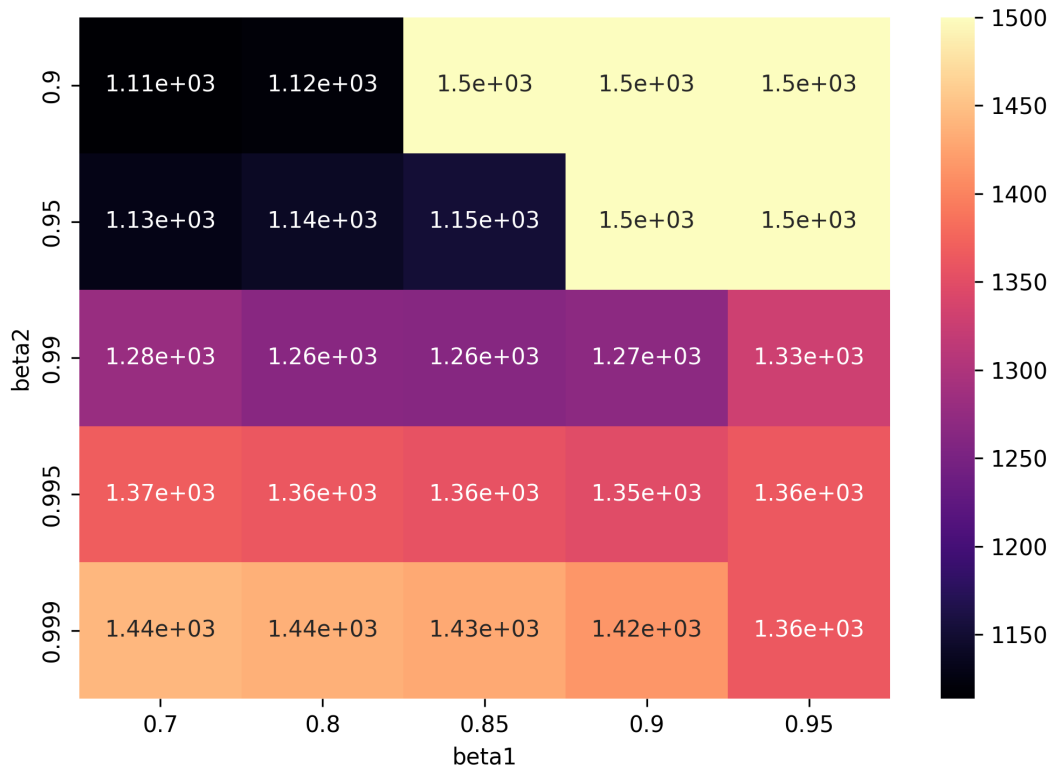


Рис. 3: Чувствительность Adam к параметрам β_1 и β_2

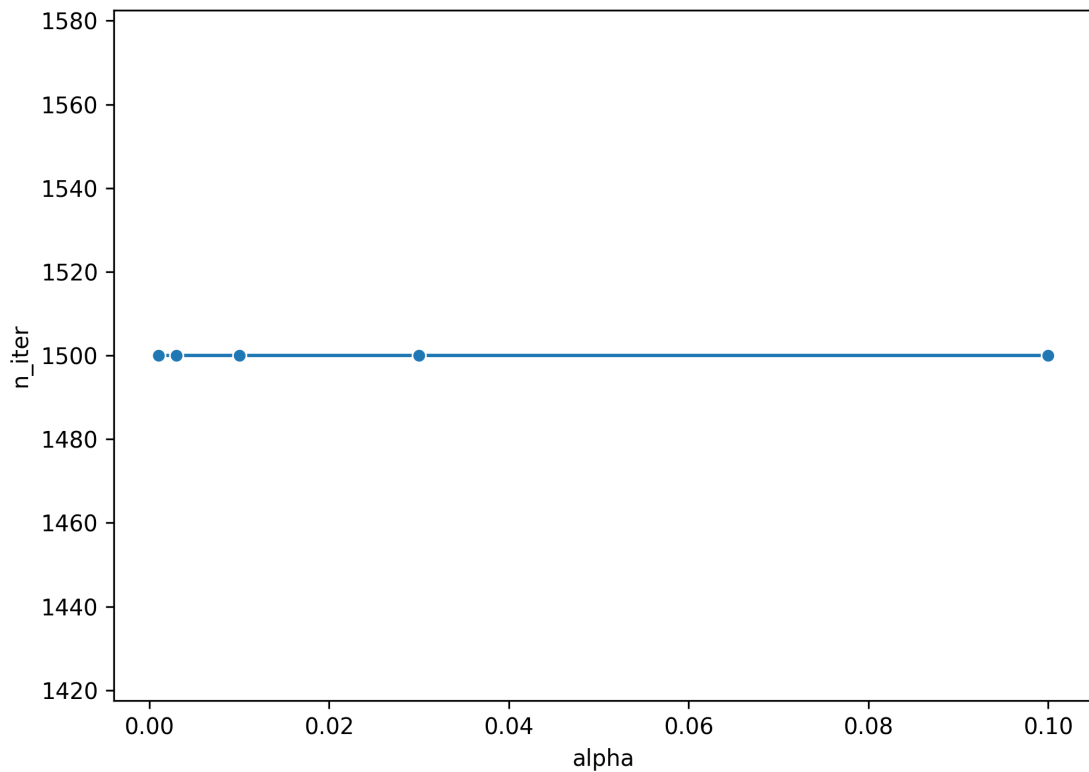


Рис. 4: Чувствительность AdaGrad к начальному шагу

Промежуточный вывод. Для инерционных методов важна согласованность шага и коэффициента инерции: α задает силу текущего градиента, а β определяет, сколько прошлой скорости переносится в новый шаг. Если оба эффекта велики, обновление может стать слишком агрессивным и начать колебаться. Для адаптивных методов аналогичную роль играет сглаживание: малое сглаживание быстро реагирует на текущий градиент, а большое делает масштаб шага стабильнее, но медленнее приспосабливается к новой области поверхности.

5.3 Трактории на квадратичных функциях

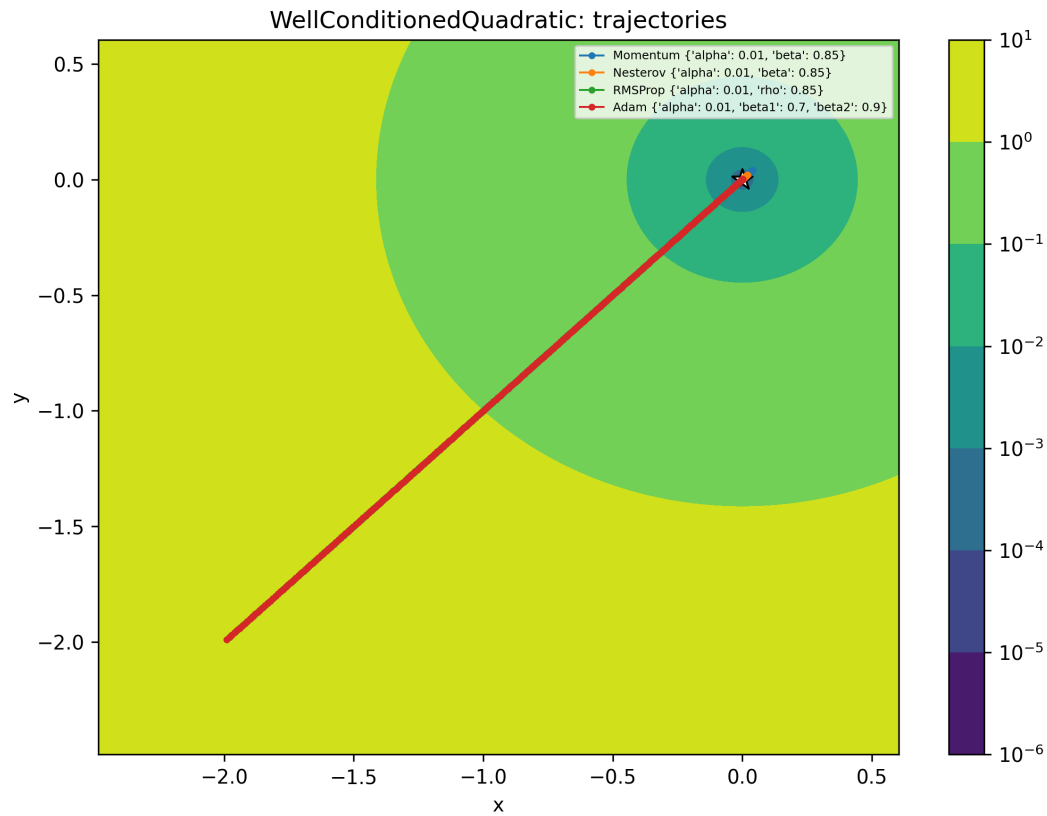


Рис. 5: Лучшие траектории методов на хорошо обусловленной квадратичной функции

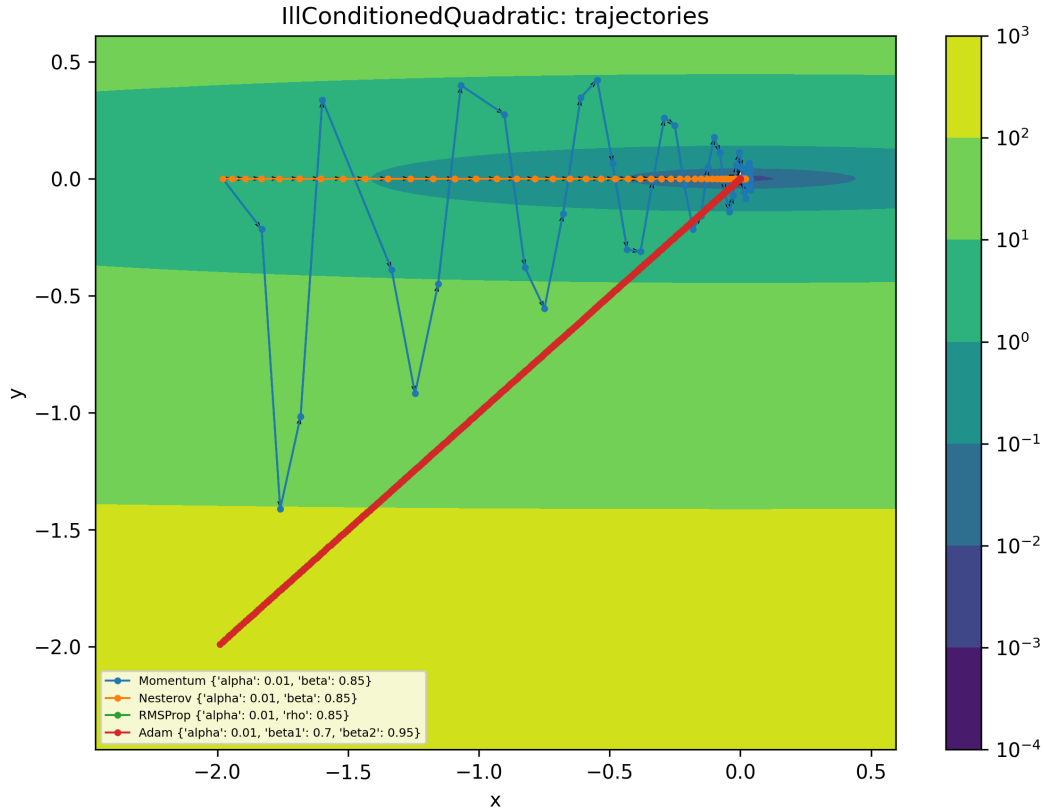


Рис. 6: Лучшие траектории методов на плохо обусловленной квадратичной функции

Промежуточный вывод. На хорошо обусловленной функции траектории в основном направлены к центру линий уровня, потому что масштаб по координатам одинаков и направление градиента хорошо согласовано с направлением к минимуму. На плохо обусловленной функции адаптивное масштабирование и инерция уменьшают влияние вытянутости разными способами: инерция накапливает движение вдоль устойчивого направления, а RMSProp и Adam уменьшают шаг по координатам с большими недавними градиентами. Поэтому они выигрывают у простого градиентного спуска с малым шагом.

5.4 Сложные функции

В таблице приведены лучшие результаты по значению функции среди исследованных параметров и стартовых точек.

Таблица 2: Лучшие запуски на сложных функциях

Функция	Метод	Лучший набор параметров	Итерации
Ackley	Momentum	$\alpha = 3 \cdot 10^{-4}, \beta = 0.95$	562
Ackley	Nesterov	$\alpha = 3 \cdot 10^{-3}, \beta = 0.50$	43
Ackley	AdaGrad	$\alpha = 10^{-2}$	81
Ackley	RMSProp	$\alpha = 10^{-4}, \rho = 0.90$	376
Ackley	AdaDelta	$\rho = 0.85$	175
Ackley	Adam	$\alpha = 3 \cdot 10^{-4}, \beta_1 = 0.70, \beta_2 = 0.95$	230
Rosenbrock	Momentum	$\alpha = 3 \cdot 10^{-3}, \beta = 0.90$	1277
Rosenbrock	Nesterov	$\alpha = 10^{-3}, \beta = 0.95$	1500
Rosenbrock	Adam	$\alpha = 10^{-2}, \beta_1 = 0.85, \beta_2 = 0.99$	1500
Himmelblau	Momentum	$\alpha = 3 \cdot 10^{-3}, \beta = 0.50$	63
Himmelblau	Nesterov	$\alpha = 10^{-2}, \beta = 0.50$	24
Himmelblau	Adam	$\alpha = 10^{-3}, \beta_1 = 0.85, \beta_2 = 0.99$	1339

Промежуточный вывод. На функции Химмельблау методы обычно находили один из глобальных минимумов, потому что после попадания в область притяжения градиентные направления достаточно уверенно ведут к ближайшему минимуму. На функции Розенброка ситуация сложнее: узкая изогнутая долина требует не только правильного масштаба шага, но и постоянного изменения направления вдоль кривой. Инерция может ускорять движение, но также может уносить поперек долины; адаптивные масштабы помогают с длиной шага, но не устраняют полностью проблему направления.

5.5 Траектории на сложных функциях

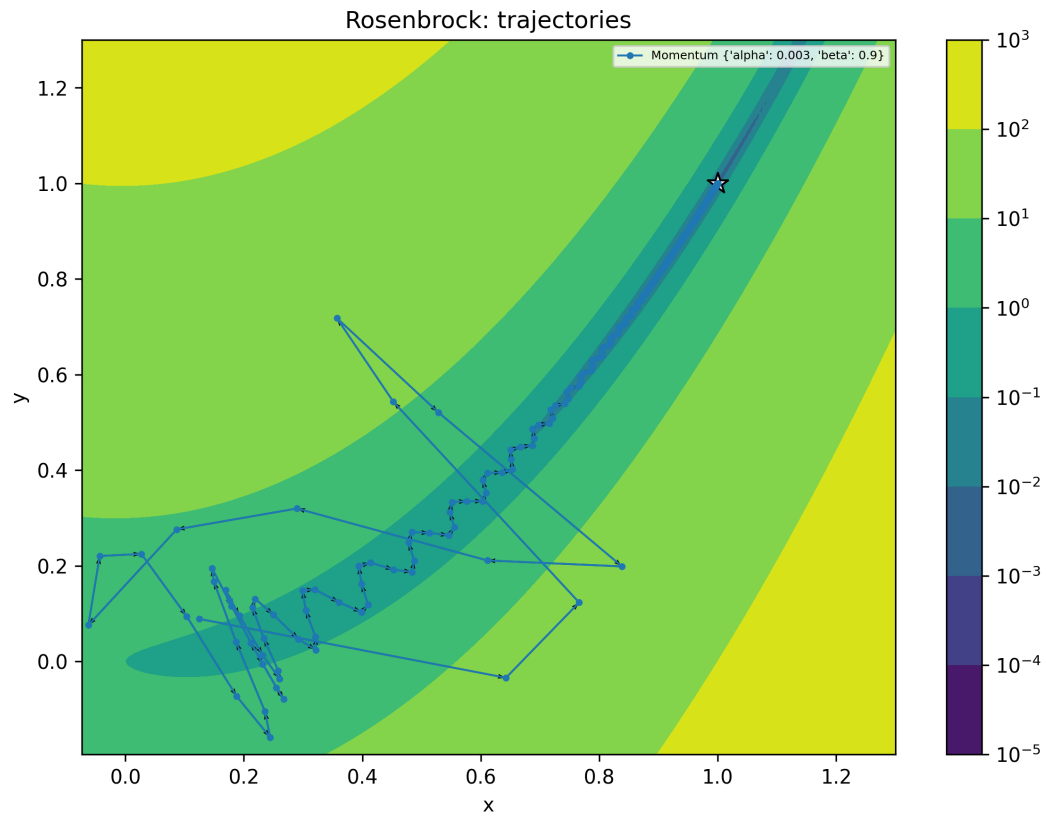


Рис. 7: Лучшие траектории методов на функции Розенброка

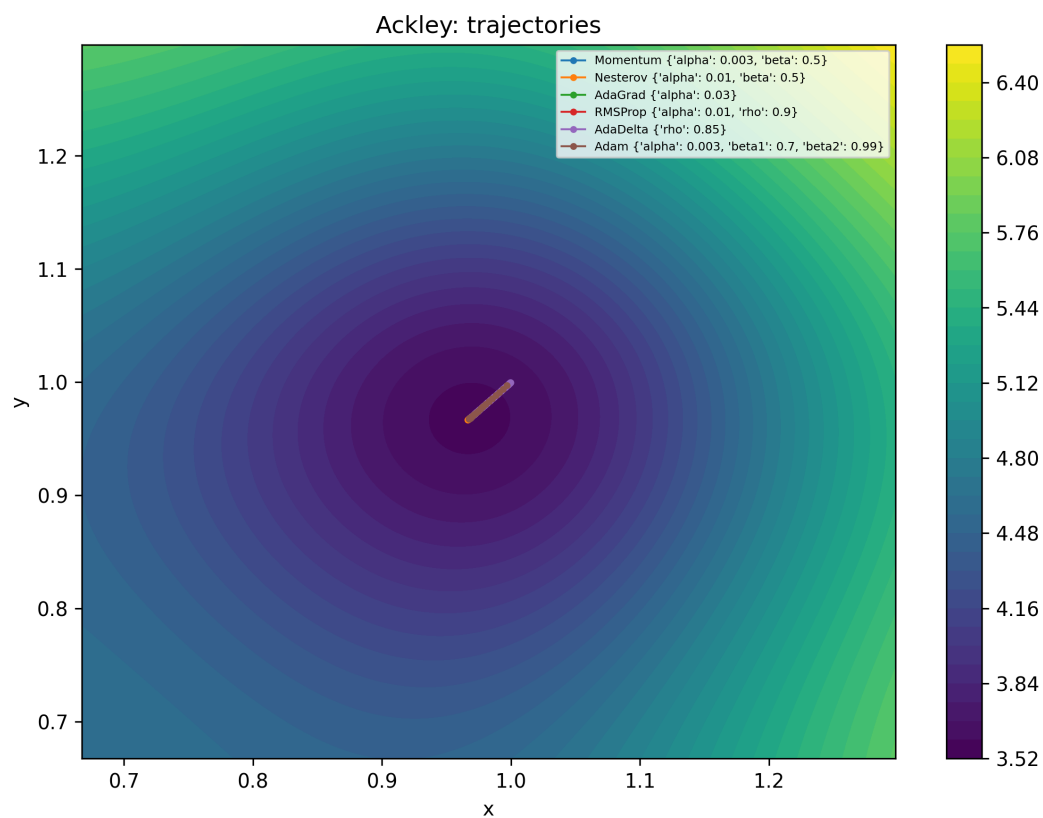


Рис. 8: Лучшие траектории методов на функции Экли

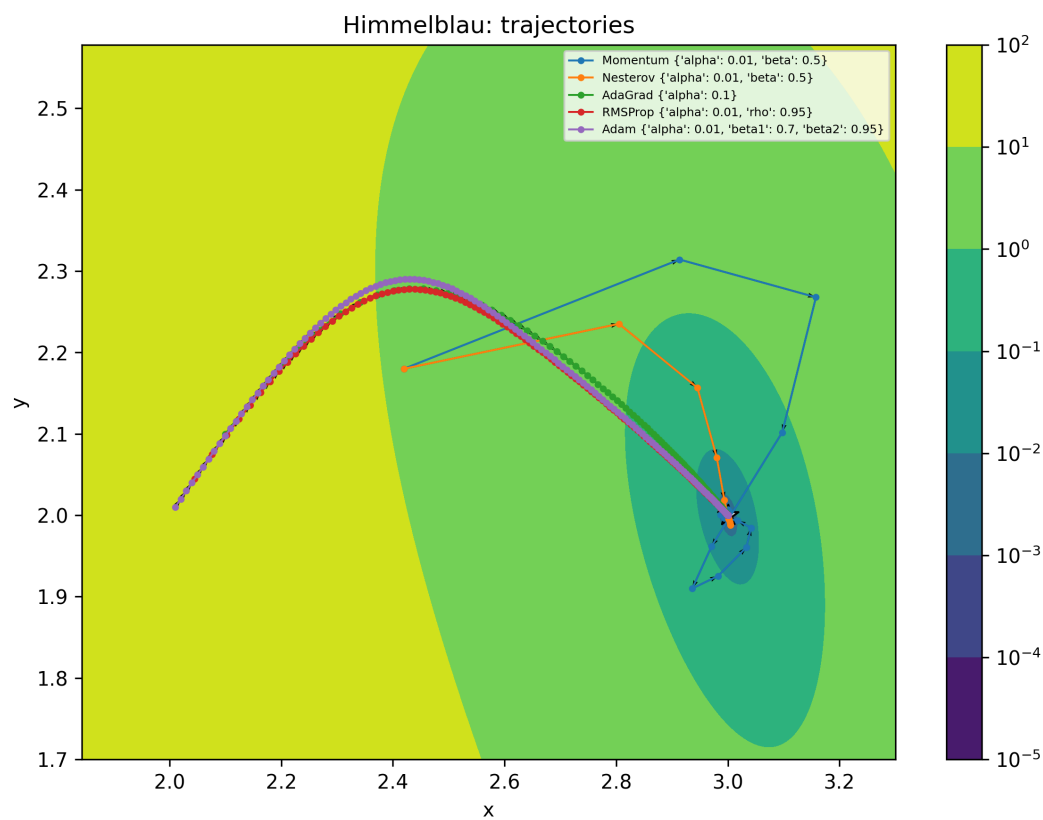


Рис. 9: Лучшие траектории методов на функции Химмельблау

Промежуточный вывод. Визуализация траекторий особенно важна для сложных функций: одинаковое итоговое число итераций может соответствовать совершенно разному поведению. У инерционных методов на графиках видны более длинные и сглаженные перемещения, поскольку шаг зависит от прошлой скорости. У адаптивных методов траектория может быть менее прямой, зато координатное масштабирование уменьшает слишком резкие движения в направлениях с большими градиентами. Поэтому график траектории дополняет таблицу и показывает, как именно внутренняя логика метода влияет на путь к минимуму.

6 Сводная таблица эксперимента

Полная сводка всех 1470 запусков сохраняется в файле `../outputs/lab3/summary.csv`. В ней содержатся функция, стартовая точка, оптимизатор, параметры запуска, признак сходимости, итоговая точка, значение функции и счетчики вычислений.

Полная таблица доступна в `../outputs/lab3/summary.csv`.

7 Общий вывод

В лабораторной работе были реализованы шесть модификаций градиентного спуска. Эксперименты показали, что добавление инерции и адаптивного масштабирования может существенно ускорить сходимость по сравнению с классическим градиентным спуском, но делает методы чувствительными к гиперпараметрам.

Momentum и Nesterov хорошо проявили себя на квадратичных функциях. Для обеих квадратичных моделей удачными оказались параметры порядка $\alpha = 10^{-2}$ и $\beta = 0.85$. Метод Нестерова иногда требовал меньше итераций на плохо обусловленной функции, однако платил за это дополнительными вычислениями градиента.

RMSProp показал устойчивую работу на квадратичных функциях и подтвердил полезность экспоненциального сглаживания квадратов градиента. AdaGrad и AdaDelta в выбранной сетке часто двигались слишком осторожно и не достигали точности до лимита итераций.

Adam оказался универсальным методом, особенно полезным как компромисс между инерцией и адаптивным шагом. При этом на детерминированных тестовых задачах он не всегда был самым быстрым: на простых квадратичных функциях Momentum и RMSProp часто требовали меньше итераций.

На сложных функциях результат сильнее зависел от стартовой точки и геометрии поверхности. Функция Розенброка осталась наиболее трудной из-за узкой изогнутой долины, а функция Химмельблау хорошо решалась из стартовых точек, попадающих в области притяжения глобальных минимумов.